

CSCE 627 THEORY OF COMPUTABILITY · MODULE 07 · LECTURE NOTES

Degrees of Unsolvability

Undecidable is not one thing.

MODULE OUTCOMES

- Define an oracle machine and relative computability.
- Explain Turing reducibility and Turing degrees.
- Place problems in the arithmetic hierarchy.
- Explain why the hierarchy does not collapse.
- Explain why this structure matters for Module 08.

1 Oracles and relative computability

AN ORACLE MACHINE CAN ASK A QUESTION IT CANNOT ANSWER

Give a Turing machine a black box that answers membership in some fixed language A , in a single step, correctly. That is an oracle machine, and it is a thought experiment rather than a device.

With a HALT oracle, the halting problem becomes trivially decidable — and new problems become undecidable *relative to that oracle*, which is the interesting part.

' B is Turing reducible to A ', written $B \leq_T A$, means an A -oracle machine decides B . This is a weaker and more general notion than Module 06's mapping reduction: the machine may consult the oracle many times, adaptively, and may invert its answers — so Turing reducibility does not distinguish a language from its complement, and mapping reducibility does.

And it gives a notion of equivalent difficulty: A and B have the same *Turing degree* if each is Turing reducible to the other — so 'as hard as' becomes a precise equivalence relation, and the equivalence classes can be partially ordered (§4).

THE HALTING PROBLEM RELATIVISES, AND THAT IS THE KEY FACT

Module 05's proof used nothing about the machine model except that programs can be encoded as data (Module 04 §3) — so it goes through unchanged for oracle machines.

So 'does this HALT-oracle machine halt on this input?' is undecidable even by a machine with a HALT oracle. Call that language HALT' , the *jump* of HALT.

And you can do it again, forever. HALT, HALT' , HALT'' , and so on — an infinite, strictly increasing sequence of difficulties, each one undecidable relative to everything below it.

So 'undecidable' names infinitely many distinct levels rather than one. Which is why Module 05's EQ_{TM} really is harder than halting — it sits strictly above it in §2's hierarchy — rather than being loosely described that way.

2 The arithmetic hierarchy

Write the problem as a formula over a DECIDABLE predicate R , and count the leading quantifiers.

Σ_1 exists y . $R(x,y)$
 RECOGNISABLE. "M halts on w " = there EXISTS a halting computation. One existential, and a finite witness.

Π_1 forall y . $R(x,y)$
 CO-RECOGNISABLE. "M halts on EVERY input" -- no finite witness exists, and the complement is Σ_1 .

Σ_2 exists y forall z . $R(x,y,z)$
 "M accepts a finite language": there EXISTS a length bound such that FOR ALL longer strings M rejects.

Π_2 forall y exists z . $R(x,y,z)$
 "M halts on infinitely many inputs".

AND SO ON, ALTERNATING. The hierarchy is STRICT: each level contains problems not in any lower level, proved by diagonalisation at that level (Module 05 section 3).

SO: TO PLACE A PROBLEM, WRITE IT OUT WITH QUANTIFIERS AND COUNT THE ALTERNATIONS. That is the whole method, and it is mechanical.

Σ_1 and Π_1 intersect in exactly the DECIDABLE problems -- which is Module 04 section 2's complement theorem, restated in this language.

Quantifier alternation is the measure of difficulty, and the same idea measures complexity in CSCE 637's polynomial hierarchy, with the quantifiers bounded to polynomial-length witnesses: Σ_1^P is NP, Π_1^P is co-NP, and the alternation continues. So this module and CSCE 637's hierarchy are the same

construction at two different resource bounds, which is worth noticing because it means the intuition transfers.

Problem	Level	Why
Primality	Decidable	$\Sigma_1 \cap \Pi_1$
M halts on w	Σ_1	There exists a halting run.
M halts on all inputs	Π_2	For all w, there exists a run that halts.
M accepts nothing	Π_1	For all w, M does not accept w.
M accepts a finite set	Σ_2	There exists a bound, for all longer strings M rejects.
M_1 and M_2 are equivalent	Π_2	For all w, the two agree — and each agreement is itself Σ_1 .

Program equivalence is Π_2 , strictly above halting — **which is the precise form of the intuition that 'do these two programs do the same thing' is a harder question than 'does this program finish'**. The intuition was correct and the hierarchy says exactly how much.

3 What the hierarchy buys

- It distinguishes 'undecidable' from 'hopeless'. A Σ_1 problem has verifiable yes-instances, so a semi-decision procedure exists and can be genuinely useful — run it, and if it answers you have a proof. A Π_2 problem has no such procedure in either direction.
- It predicts which direction a tool can be sound in. A Π_1 property can be refuted by a single counterexample and never confirmed by finitely much evidence — which is exactly the asymmetry of testing, and is why 'tests cannot prove the absence of bugs' is a theorem rather than a slogan.
- It explains why some verification is harder than other verification. Safety is Π_1 ; liveness is Π_2 — and liveness genuinely is harder, in a provable sense (see the callout).
- It is where the polynomial hierarchy comes from (CSCE 637), by bounding the quantifiers to polynomial-length witnesses — so the structure recurs with resources attached.
- And it tells you that hard problems are not all equally hard — Post's problem asked whether any recognisable problem lies strictly between decidable and halting, and the answer is yes (§4).

SAFETY AND LIVENESS, AND WHY THE ASYMMETRY IS REAL

A safety property says 'nothing bad ever happens' — for all reachable states, the state is good. That is Π_1 : a single bad state refutes it, and no finite amount of good behaviour confirms it.

A liveness property says 'something good eventually happens' — for all executions, there exists a point at which it holds. That is Π_2 , strictly harder.

Which is why testing finds safety violations and essentially never establishes liveness. A failing test is a complete counterexample to a safety property, and no test of any length is evidence about eventuality — the run you observed simply had not got there yet, and you cannot tell that from never getting there.

And it is why model checkers treat the two differently: safety by reachability analysis, and liveness by cycle detection on the product of the system with a Büchi automaton. The algorithms differ because the hierarchy levels differ, which is a case of this module's abstract structure determining a concrete tool's architecture — and it is the most useful practical consequence in the module.

4 The structure of the degrees

THE DEGREE STRUCTURE IS RICH, STRANGE, AND MOSTLY SETTLED

There is a least degree — the decidable problems — and the jump operator always strictly increases (§1), so there is no greatest degree and the structure is unbounded above.

Post's problem asked whether there is a recognisable problem whose degree lies strictly between the decidable problems and HALT. **The answer is yes — proved independently by Friedberg and Muchnik in 1956, by the *priority method***, which was invented for the purpose and became the field's central technique.

And the degrees are not a line. There are incomparable degrees — pairs of problems neither of which is Turing reducible to the other — so 'harder than' is a partial order rather than a scale, and two problems can be unsolvable in unrelated ways.

This is a deep and specialised field (computability theory proper, and it continues), and **knowing that it exists and that difficulty is a partial order rather than a single axis is the useful takeaway** — more useful than any particular result in it, and enough to stop you from asking which of two undecidable problems is harder without checking that the question has an answer.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — sections 6.3–6.4	Oracles, Turing reducibility, and the hierarchy — a brief but sufficient treatment.
Soare — Turing Computability: Theory and Applications	The modern reference for §4 , including the priority method and Post's problem.
Rogers — Theory of Recursive Functions and Effective Computability	The classical reference , and still the clearest treatment of the arithmetic hierarchy of §2. Library copy.
Baier & Katoen — Principles of Model Checking	§3's safety/liveness distinction, operationalised — the two different algorithms and why. Library copy; the lecture notes from the authors' courses are free.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Define an oracle machine** and write a program sketch using a HALT oracle to decide something interesting.

- 2 **Prove that halting relativises**, by checking Module 05's proof step by step.
- 3 **Show that Turing reducibility does not distinguish a language from its complement**, and that mapping reducibility does.
- 4 **Place six problems in the arithmetic hierarchy** by writing them with quantifiers.
- 5 **Find a Σ_2 problem of your own** and justify the level.
- 6 **Confirm $\Sigma_1 \cap \Pi_1$ is the decidable problems** and relate it to Module 04's theorem.
- 7 **Classify three properties of a system you have built** as safety or liveness, and state the level of each.
- 8 **Write a test that refutes a safety property** and explain why no test establishes the corresponding liveness one.
- 9 **Read about the priority method** and summarise what Post's problem asked.
- 10 **Find two problems you believe are incomparable** and say why you cannot easily tell.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define an oracle machine and Turing reducibility.
- 2 How does Turing reducibility differ from mapping reducibility?
- 3 Why does the halting problem relativise, and what follows?
- 4 Give the first four levels of the arithmetic hierarchy with examples.
- 5 What is the method for placing a problem?
- 6 What is $\Sigma_1 \cap \Pi_1$?
- 7 Why is program equivalence harder than halting, precisely?
- 8 Give five things the hierarchy buys.
- 9 Classify safety and liveness and explain the testing asymmetry.
- 10 What did Post's problem ask, and what is the shape of the degree order?